

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 5. Conditional Statements

Lecture: 38. Compound Conditional Statements

Section 1: Lecture Summary

This lecture covers **compound conditional statements** in Python, focusing on how to combine multiple conditions using **logical operators**. It begins with an introduction to the logical operators ``and``, ``or``, and ``not``, explaining their origins and how they are used in programming. The lecture revisits basic conditional statements and shows how logical operators create compound statements that evaluate multiple conditions together. The concept of a **truth table** is introduced to explain the behavior of these operators for all possible combinations of True and False conditions. Several examples demonstrate how these compound conditionals work in practice, followed by testing these examples in a Jupyter Notebook environment to confirm their outcomes. Finally, the lecture emphasizes the importance of mastering these concepts to avoid common programming bugs related to incorrect condition formulation.

Section 2: Key Concepts and Explanations

- Logical Operators in Python

- ``and``: True if both conditions are true; otherwise False.
- ``or``: True if at least one condition is true; false only if both are false.
- ``not``: Unary operator; reverses the truth value of a single condition.

- Compound Conditional Statements

- Formed by combining two or more relational expressions using logical operators.

- Used to check multiple conditions simultaneously.

- Relational Operators Review

- Common relational operators used in conditions: `<` , `<=` , `>` , `>=` , `==` , `!=` .
- They return a boolean value (`True` or `False`).

- Truth Tables

- Define the output of logical operators given all possible truth values of the component conditions.

Condition 1	Condition 2	`and` Result	`or` Result
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

- For `not`:

Condition	`not` Result
True	False
False	True

- Usage Scenarios

- `and`: Requires all conditions to be true for the whole statement to be true.
- `or`: Requires any one condition to be true.
- `not`: Negates one single condition, turning true to false and vice versa.

- Importance of Proper Condition Writing

- Improperly formed compound conditions cause logic bugs and debugging challenges.
- Precision in logical statements is vital in programming.

Section 3: Example Code and Use Cases

Example 1

```
a = 5
b = 7
c = 3

print(a > b and a > c) # False because a > b is False, a > c is True,
combined with 'and'
```

Explanation: `a > b` is False, `a > c` is True, using `and` returns False because both need to be True.

Example 2

```
a = 5
b = 7
c = 3

print(a > b or a > c) # True because `a > c` is True, 'or' requires only
one True
```

Explanation: `a > b` is False, `a > c` is True, using `or` returns True.

Example 3

```
a = 8

print(a > 0 and a < 10) # True, both conditions are True
```

Explanation: `a` is 8, which is greater than 0 and less than 10; both conditions True means `and` returns True.

Example 4

```
a = 8

print(not a == 8) # False - 'not' negates the condition `a == 8` which is True
```

Explanation: `a == 8` is True, `not` makes it False.

Section 4: Key Takeaways

- Compound conditional statements combine multiple conditions using `and`, `or`, and `not`.
- **`and`** is True only if all combined conditions are True.
- **`or`** is True if any one of the combined conditions is True.
- **`not`** reverses the truth value of a single condition.
- Truth tables systematically explain how logical operators evaluate based on condition values.
- Proper construction of compound conditions is essential to avoid logical errors in programming.
- Testing logical expressions, e.g., in Jupyter Notebook, helps verify expected outcomes.
- Mastery of these concepts is foundational for writing clear, bug-free conditional logic in Python.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.

